

SESSION-1

Task 1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

```
root@DUDDUKURIs: ~/skill/w × + v
root@DUDDUKURIs:~# cd skill/week-1
root@DUDDUKURIs:~/skill/week-1# nano main.c
root@DUDDUKURIs:~/skill/week-1# |
```

CODE:

```
GNU nano 7.2 main.c
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation is working successfully.\n");

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs: ~/skill/w × + v
root@DUDDUKURIs:~# cd skill/week-1
root@DUDDUKURIs:~/skill/week-1# nano main.c
root@DUDDUKURIs:~/skill/week-1# nano main.c
root@DUDDUKURIs:~/skill/week-1# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-1# ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
root@DUDDUKURIs:~/skill/week-1# |
```

```
root@DUDDUKURIs: ~/skill/w × + ∨
GNU nano 7.2
CC = gcc
CFLAGS = -Wall -Wextra

all: main

main: main.c
    $(CC) $(CFLAGS) main.c -o main

clean:
    rm -f main|
```

Verifying linux environment by uname-a:

```
root@DUDDUKURIs:~/skill/week-1# uname -a
Linux DUDDUKURIs 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu
Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@DUDDUKURIs:~/skill/week-1# |
```

Checking GCC C compiler:

```
root@DUDDUKURIs:~/skill/week-1# gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
root@DUDDUKURIs:~/skill/week-1# |
```

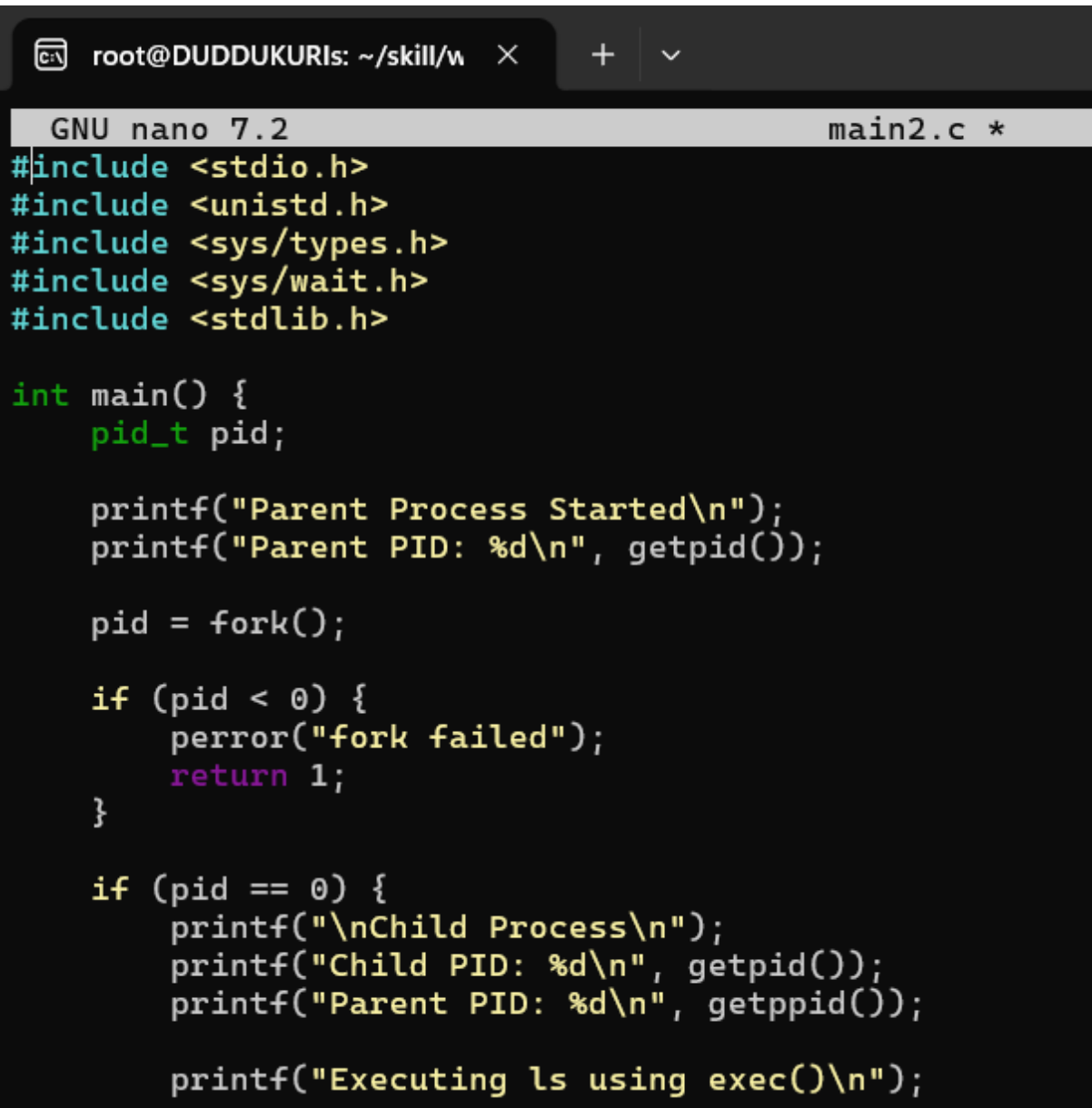
Git version:

```
root@DUDDUKURIs:~/skill/week-1# git --version
git version 2.43.0
root@DUDDUKURIs:~/skill/week-1# |
```

Task 2: To Analyse process abstraction, execute `fork()`, understand `exec()` family, analyse parent-child relationships, inspect process tree, practice system call tracing.

```
root@DUDDUKURIs:~/skill/week-1#  
root@DUDDUKURIs:~/skill/week-1# nano main2.c  
root@DUDDUKURIs:~/skill/week-1# |
```

CODE:



```
GNU nano 7.2 main2.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main() {  
    pid_t pid;  
  
    printf("Parent Process Started\n");  
    printf("Parent PID: %d\n", getpid());  
  
    pid = fork();  
  
    if (pid < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (pid == 0) {  
        printf("\nChild Process\n");  
        printf("Child PID: %d\n", getpid());  
        printf("Parent PID: %d\n", getppid());  
  
        printf("Executing ls using exec()\n");
```

```

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
|

```

OUTPUT:

```

root@DUDDUKURIs:~/skill/week-1# nano main2.c
root@DUDDUKURIs:~/skill/week-1# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-1# ./main2
Parent Process Started
Parent PID: 604

Parent Process
Parent PID: 604
Child PID: 605
Parent waiting for child...

Child Process
Child PID: 605
Parent PID: 604
Executing ls using exec()
total 28
-rw-r--r-- 1 root root 109 Aug 15 02:46 Makefile
-rw-r--r-- 1 root root 151 Aug 15 02:43 main.c
-rwxr-xr-x 1 root root 16296 Aug 15 02:57 main2
-rw-r--r-- 1 root root 879 Aug 15 02:56 main2.c
Child process completed.
root@DUDDUKURIs:~/skill/week-1# |

```

SESSION-2

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

A) Create a file by using

```
root@DUDDUKURIs: ~/skill/week-3

root@DUDDUKURIs:~# cd skill
root@DUDDUKURIs:~/skill# cd week-3
root@DUDDUKURIs:~/skill/week-3# nano main.c
root@DUDDUKURIs:~/skill/week-3#
```

Then press enter and the code terminal will open then write a code

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        // Remove newline
        input[strcspn(input, "\n")] = 0;

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting Shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

After writing the code save the your program by ctrl O and for coming back to command press ctrl X then compile it by the command

```
root@DUDDUKURIs: ~/skill/week-3

root@DUDDUKURIs:~# cd skill
root@DUDDUKURIs:~/skill# cd week-3
root@DUDDUKURIs:~/skill/week-3# nano main.c
root@DUDDUKURIs:~/skill/week-3# gcc main.c -o main_
```

After compiling to get the output enter command

```
root@DUDDUKURIs: ~/skill/week-3
root@DUDDUKURIs:~# cd skill
root@DUDDUKURIs:~/skill# cd week-3
root@DUDDUKURIs:~/skill/week-3# nano main.c
root@DUDDUKURIs:~/skill/week-3# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-3# ./main
Enter command (type exit to quit): ls
You entered: ls
Enter command (type exit to quit): O SSP
You entered: O SSP
Enter command (type exit to quit): _
```

Task 2: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
root@DUDDUKURIs: ~/skill/w  X + v
root@DUDDUKURIs:~# cd skill/week-1
root@DUDDUKURIs:~/skill/week-1# nano main2.c
root@DUDDUKURIs:~/skill/week-1# gcc main2.c -o main2
```

CODE:

```
root@DUDDUKURIs: ~/skill/w × + v
GNU nano 7.2
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input> ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }

    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}
```

```
root@DUDDUKURIs: ~/skill/w × + v
root@DUDDUKURIs:~# cd skill/week-1
root@DUDDUKURIs:~/skill/week-1# nano main2.c
root@DUDDUKURIs:~/skill/week-1# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-1# ./main2
Type a command and press Enter:
input> hello world

Command entered: hello world
Characters stored: 11
root@DUDDUKURIs:~/skill/week-1# |
```